

SISTEMAS OPERATIVOS

Conferencia 6 · Tema V: Administración de Memoria

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Comprender los mecanismos de administración de memoria en los sistemas operativos: monoprogramación, protección, relocalización, swapping, multiprogramación con particiones fijas y variables, técnicas y algoritmos de asignación, paginación, segmentación y esquemas combinados; relacionándolos con las implementaciones en sistemas operativos actuales.

Sumario

- Aspectos introductorios. Jerarquía de memoria.
- Uso de la memoria en monoprogramación.
- Protección y relocalización estática y dinámica.
- Intercambio de trabajos (Swapping). Solapamiento.
- Multiprogramación con particiones fijas (MFT). Fragmentación interna.
- Multiprogramación con particiones variables (MVT). Fragmentación externa. Compactación.
- Técnicas de gestión del espacio libre: Mapa de bits, listas enlazadas, Buddy System.
- Algoritmos de asignación: First Fit, Next Fit, Best Fit, Worst Fit.
- Paginación: traducción de direcciones, MMU, TLB, páginas compartidas.
- Segmentación: tabla de segmentos, fragmentación, protección y compartición.
- Esquemas combinados: segmentación paginada y paginación segmentada.
- Administración de memoria en SO actuales: Linux, Windows.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos, 4ª ed. Págs. 85–160.
- Silberschatz, Galvin, Gagne. Operating System Concepts, 10ª ed. Caps. 8–9.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 191–206.
- Love, R. Linux Kernel Development, 3ª ed. Caps. 11–12 (Memory Management).

1. Aspectos Introductorios

La memoria es uno de los recursos más importantes de la computadora y debe ser asignada y utilizada cuidadosamente. El componente del sistema operativo encargado de esta gestión se denomina administrador de memoria (Memory Manager).

Existen diferentes esquemas de administración de memoria cuya complejidad y capacidad depende en gran medida de las posibilidades que brinde el hardware subyacente. La evolución histórica va desde la ausencia total de gestión (hasta ~1960) hasta los sofisticados esquemas de memoria virtual actuales.

1.1 Jerarquía de Memoria

La memoria del sistema se organiza en una jerarquía según velocidad, capacidad y coste:

Velocidad (más rápida) ↓	Capacidad (menor) ↓	Coste/byte (mayor) ↓
Registros del CPU	< 1 KB	< 1 ns
Caché L1/L2/L3	32 KB-32 MB	1-10 ns
Memoria Principal (RAM)	4-256 GB	~80 ns
Almacenamiento SSD/NVMe	256 GB-4 TB	0.1 ms
Disco Duro (HDD)	1-20 TB	5-15 ms
↑ (más lenta)	↑ (mayor)	↑ (menor)

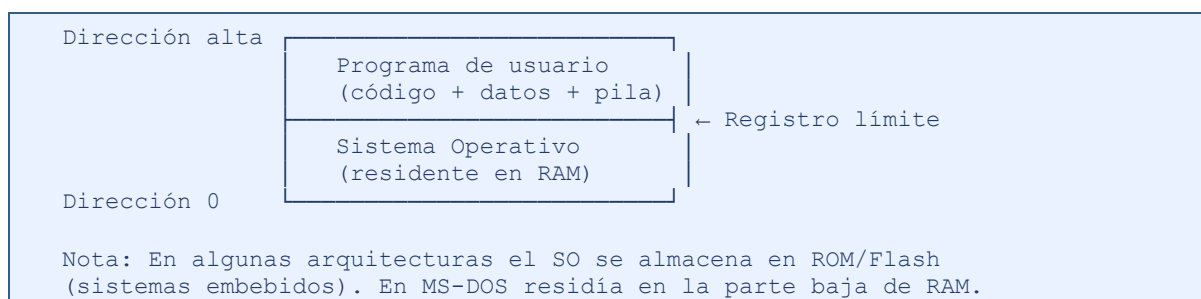
El administrador de memoria gestiona principalmente la RAM (memoria principal), intentando mantener en ella los datos e instrucciones que los procesos necesitan activamente, mientras que el resto reside en almacenamiento secundario (base de la memoria virtual).

🔴 **Nota actualizada:** En sistemas modernos de 64 bits, el espacio de direcciones virtual es de 2^{64} bytes (~18 exabytes), mucho mayor que cualquier RAM disponible. El administrador de memoria crea la ilusión de que cada proceso dispone de este enorme espacio, aunque la RAM física sea de solo 8–64 GB. Esta abstracción es la base de la memoria virtual estudiada en la siguiente conferencia.

2. Monoprogramación: Protección y Relocalización

2.1 Uso de la Memoria en Monoprogramación

El esquema más simple divide la memoria en dos partes: una para el SO (parte baja) y otra para el único programa de usuario (parte alta). Esta estructura fue utilizada en las primeras microcomputadoras (CP/M, MS-DOS).



2.2 Protección

La coexistencia del SO y al menos un programa usuario en memoria introduce la necesidad de protección: el programa usuario no debe poder acceder ni modificar el área del SO. El mecanismo clásico es un registro límite (fence register) que contiene la dirección más alta del SO. Cualquier acceso a esa zona o inferior genera una trampa.

En sistemas multiprogramados se extiende a dos registros: base y límite, que delimitan el área válida del proceso en ejecución. Las instrucciones para modificar estos registros son privilegiadas (solo ejecutables en modo kernel).

2.3 Relocalización Estática y Dinámica

Tipo	Cómo funciona	Ventajas / Desventajas
Direcciones absolutas	El compilador genera direcciones físicas fijas. El programa debe cargarse siempre en la misma posición.	Rígido. No permite multiprogramación real. No usado actualmente.
Relocalización estática	El cargador sume la dirección base a todas las referencias en el momento de la carga. Se modifica el código/datos en disco.	El programa no puede moverse en RAM tras la carga. Un solo pase.
Relocalización dinámica	El hardware suma el registro base a cada dirección lógica generada por la CPU. El programa no se modifica.	El proceso puede moverse en RAM en cualquier momento. Requiere MMU hardware.

Relocalización dinámica con registro base:

```
Instrucción: LOAD R1, 500    ← dirección LÓGICA = 500
Registro base = 14000
          |
          v (suma hardware en la MMU)
Dirección FÍSICA = 14000 + 500 = 14500
```

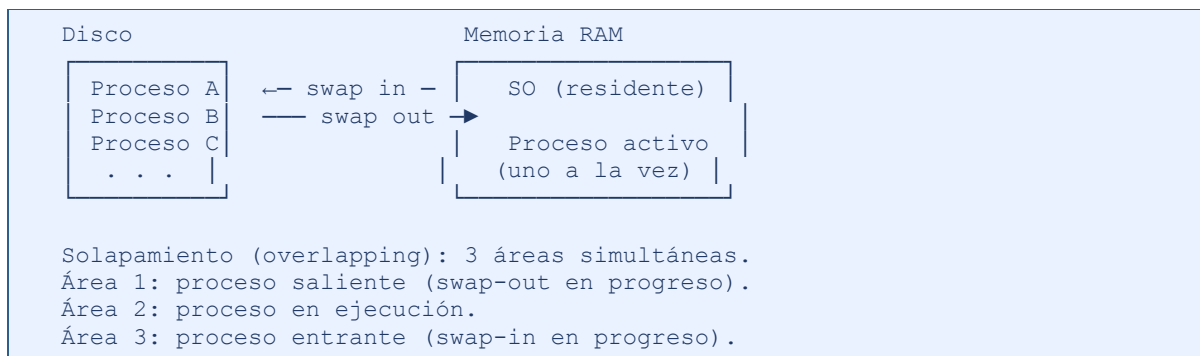
Si el proceso se mueve a la dirección 20000:
 → Solo se actualiza el registro base a 20000.
 → No hay que modificar el código del proceso.

🔴 **Nota actualizada:** En los procesadores x86 de 32 bits, los registros de segmento CS, DS, SS, ES implementaban la relocalización dinámica mediante descriptores de segmento en la GDT/LDT. En la arquitectura x86-64 (64 bits), la segmentación está prácticamente desactivada y la relocalización se realiza a través de la paginación con MMU.

3. Intercambio de Trabajos (Swapping) y Multiprogramación

3.1 Swapping

El swapping (intercambio de trabajos) permite implementar tiempo compartido incluso sin multiprogramación completa. Un trabajo se carga en memoria para ejecutarse; cuando termina su quantum, se bloquea o necesita esperar, su imagen completa se transfiere a disco y se carga el siguiente.

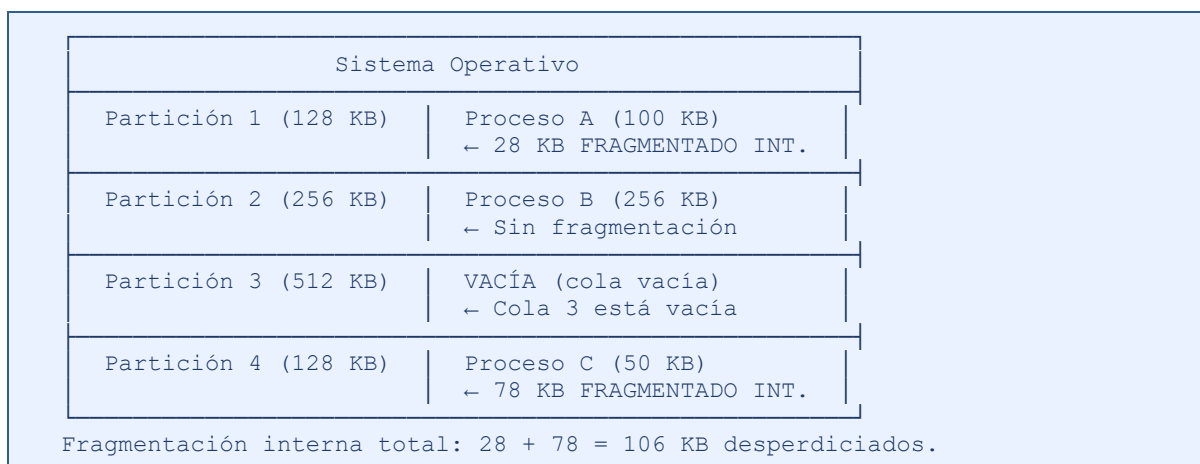


El solapamiento (overlapping) mejora el rendimiento dividiendo la memoria en 3 áreas: mientras un proceso se ejecuta, el SO transfiere al disco el que terminó y carga desde disco el siguiente, de forma simultánea gracias al DMA.

💡 **Concepto clave:** El swapping completo es costoso en tiempo (~100 MB/s para HDD → transferir 100 MB tarda ~1 seg). Por eso los SO modernos solo hacen swap de páginas individuales (demanda de páginas), no de procesos completos. Sin embargo, en dispositivos con poca RAM (Android con zswap, Linux con swap partition), el swapping por páginas sigue siendo fundamental.

3.2 Multiprogramación con Particiones Fijas (MFT)

La memoria se divide en N particiones de tamaño fijo definidas al arrancar el sistema. Cada proceso ocupa exactamente una partición. El grado de multiprogramación lo determina N.

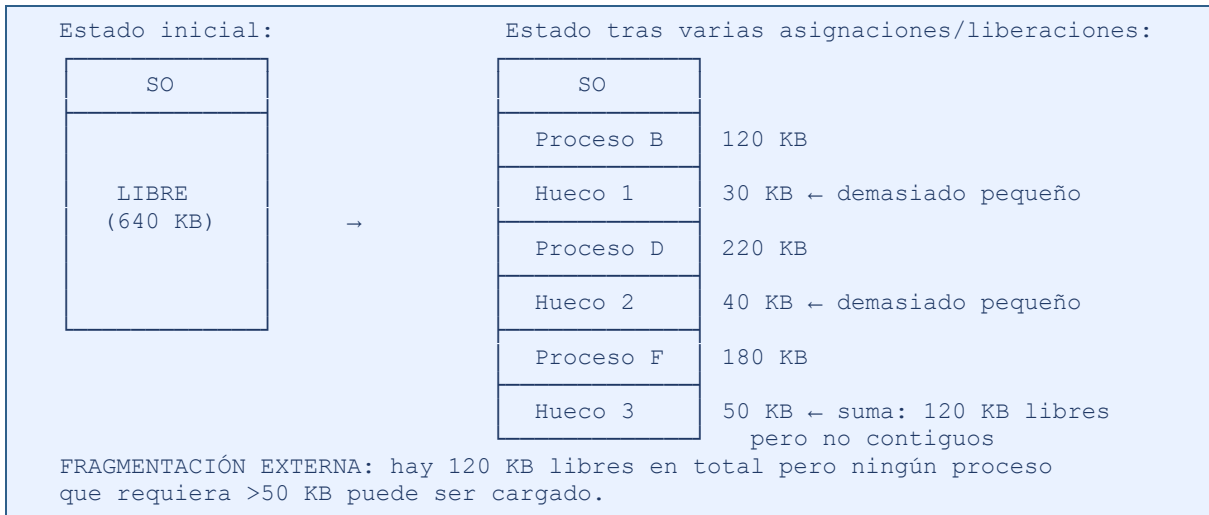


Característica	MFT — Particiones Fijas
Nº de particiones	Fijo, definido en el arranque del sistema.
Colas de entrada	Una por partición, o una global (más flexible).
Protección	Registros base+límite o bits de clave (IBM/360).

Característica	MFT — Particiones Fijas
Fragmentación	Interna: espacio desperdiciado dentro de la partición.
Ejemplo histórico	OS/MFT de IBM System/360.

3.3 Multiprogramación con Particiones Variables (MVT)

Las particiones no son predefinidas; se crean dinámicamente según las necesidades de cada proceso. Cada proceso recibe exactamente la memoria que solicita. No existe fragmentación interna, pero sí fragmentación externa con el paso del tiempo.



Compactación: técnica para resolver la fragmentación externa moviendo todos los procesos hacia un extremo de la memoria, consolidando todo el espacio libre en un único hueco. Requiere relocalización dinámica. Es costosa en tiempo de CPU y raramente se usa en la práctica.

Aspecto	MFT (Particiones Fijas)	MVT (Particiones Variables)
Fragmentación	Interna (dentro de la partición)	Externa (huecos dispersos entre procesos)
Flexibilidad	Baja — tamaños predefinidos	Alta — tamaño exacto por proceso
Complejidad del SO	Baja	Mayor (gestión de huecos)
Utilización de memoria	Peor (fragmentación interna)	Mejor (sin fragmentación interna)
Uso histórico	IBM OS/MFT (años 60–70)	Base de todos los SO modernos

4. Técnicas de Gestión del Espacio Libre y Algoritmos de Asignación

El SO debe mantener una estructura de datos que registre qué bloques de memoria están libres (huecos) y cuáles están ocupados. Existen tres técnicas fundamentales:

4.1 Mapa de Bits (Bit Map)

La memoria se divide en unidades asignables (típicamente 4 KB = un frame de página). Cada unidad se representa con 1 bit: 0 = libre, 1 = ocupado.

Memoria (dividida en unidades de 4 KB):

SO	SO	A	A	A	LIBRE	LIBRE	B	B	LIBRE	C	C	
----	----	---	---	---	-------	-------	---	---	-------	---	---	--

Unidad: 0 1 2 3 4 5 6 7 8 9 10 11

Mapa de bits correspondiente:

```
[1][1][1][1][1][0][0][1][1][0][1][1]
0 1 2 3 4 5 6 7 8 9 10 11
```

Para asignar 2 unidades contiguas: buscar '00' en el mapa → posiciones 5-6.

Overhead del mapa: 1 bit por unidad = 1 MB de RAM → 256 bytes de mapa.

Aspecto	Mapa de Bits
Ventaja	Estructura compacta y sencilla. Fácil de implementar.
Desventaja	Búsqueda de K bits consecutivos en 0 es lenta (O(N)). Poco usado en producción.
Tamaño del mapa	Con unidades de 4 KB: 1 GB de RAM → 32 KB de mapa (0,003% overhead).

4.2 Listas Enlazadas

Se mantiene una lista de bloques (ocupados y libres) ordenada por dirección ascendente. Cada nodo almacena: tipo (proceso/hueco), dirección inicio, longitud, puntero al siguiente.

Lista enlazada de bloques (P=proceso, H=hueco):

```
[P|0|200KB|→] → [H|200|50KB|→] → [P|250|300KB|→] → [H|550|100KB|→] → NULL
A (ocup.)      Hueco 1          B (ocup.)      Hueco 2
```

Al liberar proceso A: el bloque A pasa a hueco.

→ Comprobar fusión con vecinos: si Hueco_izq o Hueco_der son adyacentes, fusionar en un único hueco más grande.

Optimización: listas separadas para libres (huecos) y ocupados.

→ Búsqueda de huecos más rápida.

→ Liberación más lenta (hay que buscar vecinos en la lista de libres).

4.3 Sistema Camarada (Buddy System)

Mantiene listas de huecos libres de tamaños potencias de 2: 1, 2, 4, 8, ..., 2^n KB. Cuando se solicita memoria de tamaño k, se calcula la menor potencia de $2 \geq k$. Si hay un bloque de ese tamaño disponible se asigna; si no, se divide (split) el bloque del doble de tamaño.

Memoria total: 1 MB = 1024 KB. Listas iniciales:

Lista 1KB:[] 2KB:[] 4KB:[] ... 512KB:[] 1024KB:[LIBRE]

Solicitud de 100 KB → potencia de 2 más cercana = 128 KB:

No hay bloque de 128 KB → dividir 256 KB → 128+128

No hay 256 KB → dividir 512 KB → 256+256

No hay 512 KB → dividir 1024 KB → 512+512

Estado tras asignación de 100 KB (bloque de 128 KB asignado):
 512KB:[LIBRE] 256KB:[LIBRE] 128KB:[LIBRE] → asignado a proceso

Liberación: el bloque liberado busca su 'camarada' (buddy).
 Si el camarada está LIBRE → fusionar → bloque de 256 KB.
 Repetir recursivamente hasta que el camarada esté ocupado.

Encontrar el camarada de un bloque en dirección X de tamaño S:
 $buddy(X, S) = X \oplus S$ (operación bit a bit sencilla y rápida)

⚠ Importante: Fragmentación interna del Buddy System: para solicitudes que no son potencia de 2 el desperdicio puede ser hasta del 50% (ej: solicitar 33 KB → se asignan 64 KB → 31 KB desperdiciados). Linux usa el Buddy System para gestionar páginas físicas (ver /proc/buddyinfo) combinado con el SLAB allocator para objetos del kernel de tamaño fijo.

4.4 Algoritmos de Asignación de Memoria

Algoritmo	Descripción	Rendimiento y efecto
First Fit (Primer Ajuste)	Busca desde el inicio de la lista y toma el primer hueco $\geq$ tamaño solicitado. Divide si sobra espacio.	Rápido. Tiende a fragmentar el inicio de la lista. Generalmente el más rápido en tiempo.
Next Fit (Próximo Ajuste)	Como First Fit pero retoma la búsqueda donde se detuvo la vez anterior (puntero circular).	Distribuye la fragmentación por toda la lista. Ligeramente peor que First Fit.
Best Fit (Mejor Ajuste)	Recorre toda la lista y toma el hueco más pequeño que sea $\geq$ al tamaño solicitado.	Más lento ($O(N)$). Genera muchos huecos residuales muy pequeños (inútiles).
Worst Fit (Peor Ajuste)	Busca el hueco más grande disponible y asigna desde él.	Lento ($O(N)$). Simulaciones muestran que es el peor algoritmo. Evitar.

📁 Ejemplo: Huecos disponibles: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB. Solicitud: 212 KB.
 • First Fit: toma el hueco de 500 KB → queda un residuo de 288 KB.
 • Next Fit: depende del puntero actual.
 • Best Fit: toma el hueco de 300 KB → queda un residuo de 88 KB (el más ajustado).
 • Worst Fit: toma el hueco de 600 KB → queda un residuo de 388 KB (el mayor).

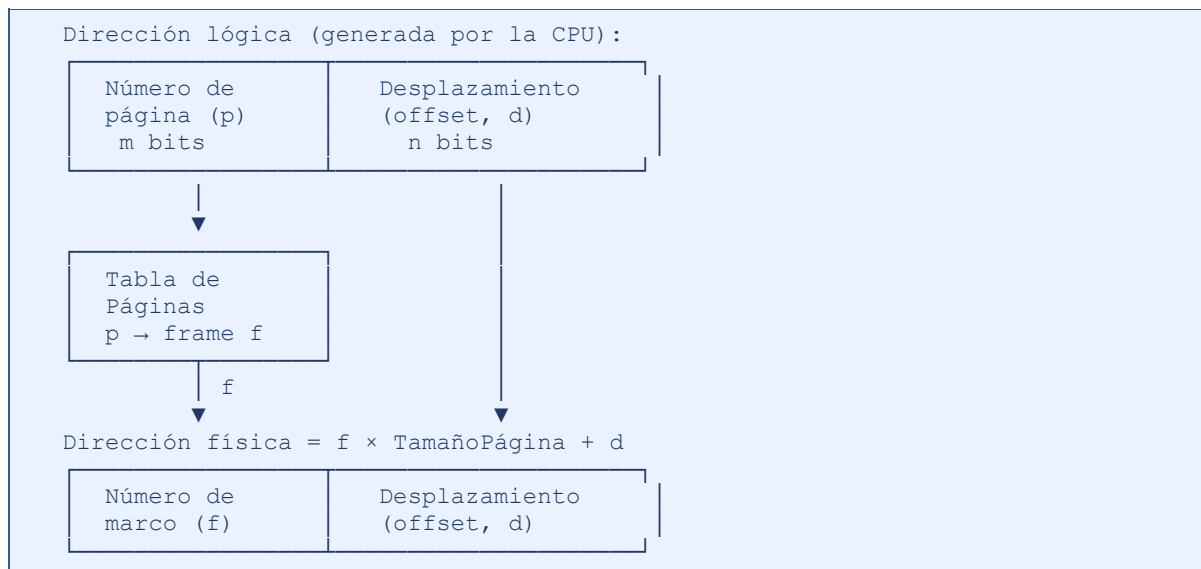
🔴 Nota actualizada: Simulaciones y análisis empíricos (Knuth, 1973) demuestran que First Fit y Next Fit son generalmente superiores a Best Fit en términos de velocidad y aprovechamiento de memoria. En el kernel de Linux, la gestión de memoria para el espacio de usuario se realiza con el algoritmo de la biblioteca glibc (ptmalloc2), que combina bins de tamaño fijo con un 'wilderness chunk' para la región contigua al heap.

5. Paginación

La paginación resuelve el problema de la fragmentación externa permitiendo que la memoria lógica de un proceso esté distribuida en cualquier parte de la memoria física, sin necesidad de ser contigua.

Principio: la memoria lógica del proceso se divide en bloques de tamaño fijo llamados páginas. La memoria física se divide en bloques del mismo tamaño llamados marcos de página (frames). Cualquier página puede cargarse en cualquier marco disponible.

5.1 Traducción de Direcciones Lógicas a Físicas



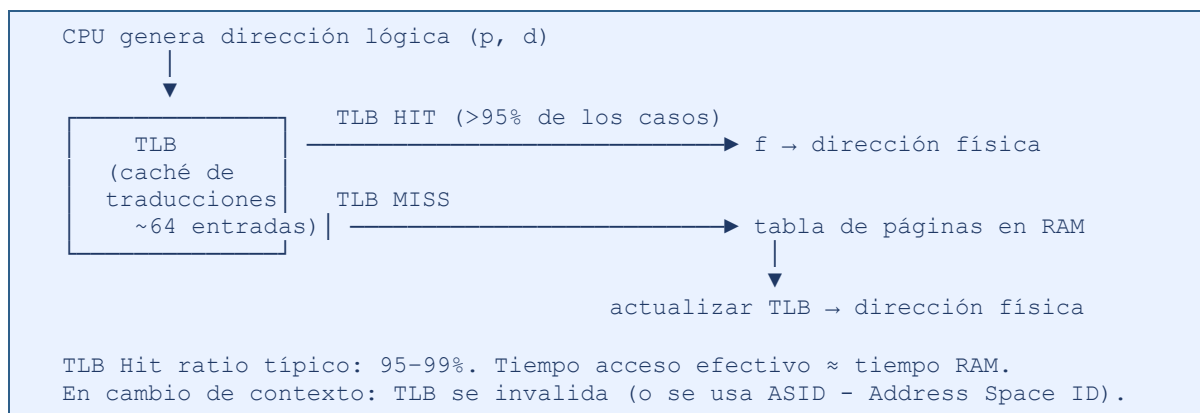
 **Ejemplo:** Tamaño de página = 4 KB = 4096 bytes. Dirección lógica = 13000. • Número de página: $p = \lfloor 13000 / 4096 \rfloor = 3$ • Desplazamiento: $d = 13000 \bmod 4096 = 712$ • Si la tabla de páginas indica que página 3 → marco 5: • Dirección física = $5 \times 4096 + 712 = 21192$

5.2 Estructura de la Tabla de Páginas

Campo de la entrada	Significado y uso
Número de marco (frame)	Dirección base del marco físico donde está almacenada la página.
Bit de validez (Present/Absent)	1 = la página está en RAM; 0 = no está (page fault si se accede).
Bits de protección	R (read), W (write), X (execute). Permiten protección por página.
Bit de modificación (Dirty)	1 = la página fue escrita desde que se cargó. Necesario para el reemplazo en memoria virtual.
Bit de referencia (Accessed)	1 = la página fue accedida recientemente. Usado por algoritmos de reemplazo (LRU, Clock).
Bit de caché (Cache disable)	Indica si la página debe ser cacheada (útil para I/O mapeada en memoria).

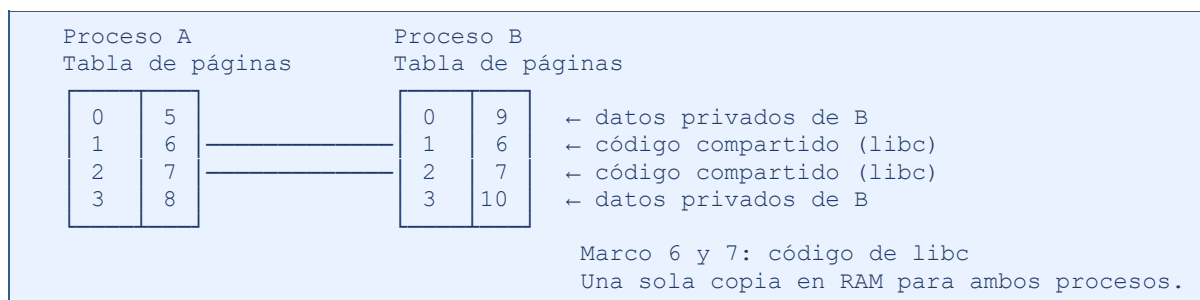
5.3 Translation Lookaside Buffer (TLB)

La tabla de páginas reside en RAM. Cada acceso a memoria requeriría 2 accesos a RAM: uno para leer la tabla y otro para acceder al dato real. El TLB es una caché hardware de alta velocidad que almacena las traducciones de páginas recientes, reduciendo el coste a 1 acceso en la mayoría de los casos.



5.4 Páginas Compartidas y Protección

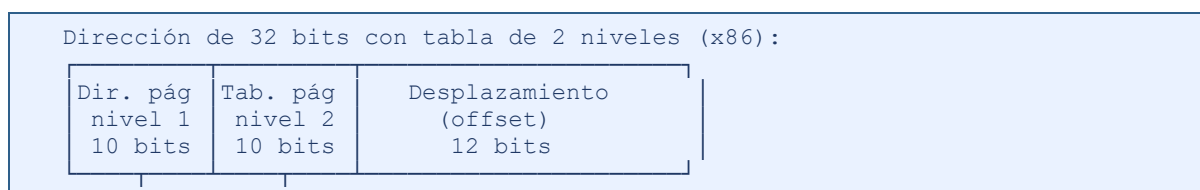
El paginado facilita la compartición de código entre procesos: basta con que las entradas de sus tablas de páginas apunten a los mismos marcos físicos. El código compartido debe ser reentrante (puro): no se puede auto-modificar durante la ejecución.



🔴 **Nota actualizada:** En Linux y Windows modernos, las páginas de código de las bibliotecas compartidas (.so / .dll) se mapean como Copy-on-Write (COW): múltiples procesos comparten el mismo marco físico de solo lectura. Si un proceso intenta escribir en él, se genera un page fault y el SO crea una copia privada para ese proceso. Esto reduce enormemente el consumo de RAM en sistemas con muchos procesos similares (ej: fork()).

5.5 Tablas de Páginas Multinivel

Para un espacio de direcciones de 32 bits con páginas de 4 KB, la tabla de páginas necesitaría $2^{20} = 1.048.576$ entradas $\times$ 4 bytes = 4 MB por proceso. Para 64 bits sería inmanejable. La solución es la tabla de páginas multinivel (jerárquica):



↓ ↓
 CR3 → Dir. tab. pág. 1 → entrada → Dir. tab. pág. 2 → marco físico

Ventaja: las tablas de nivel 2 solo se crean para las regiones del espacio de direcciones que realmente se usan (sparse address space).

x86-64 usa 4 niveles (PML4): permite 256 TB de espacio virtual.

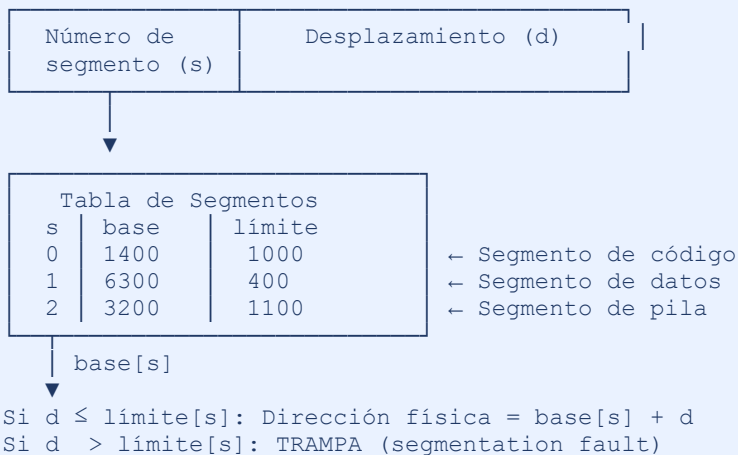
6. Segmentación

La segmentación divide el espacio de direcciones lógico en segmentos de tamaño variable, cada uno con un nombre (número) y una longitud. Se acerca más a la visión del programador: código, datos globales, pila, heap, bibliotecas como segmentos separados.

Diferencia clave: en la paginación, las páginas tienen tamaño fijo y el programador no las ve. En la segmentación, los segmentos tienen tamaño variable y el programador (o el compilador) los define explícitamente.

6.1 Traducción de Direcciones en Segmentación

Dirección lógica: (segmento s , desplazamiento d)



Aspecto	Paginación	Segmentación
Tamaño de los bloques	Fijo (páginas = frames).	Variable (un segmento puede ser desde 1 byte hasta GBs).
Visibilidad para el programador	Transparente (el programador no la ve).	Visible: el programador/compilador define los segmentos.
Fragmentación	Interna (en el último frame).	Externa (huecos de tamaño variable entre segmentos).
Compartición	Fácil (apuntar al mismo frame).	Posible pero más compleja.
Protección	Por página (bits R/W/X).	Por segmento (bits R/W/X + pertenencia).

Aspecto	Paginación	Segmentación
Complejidad hardware	MMU con tabla de páginas.	MMU con tabla de segmentos + verificación de límite.

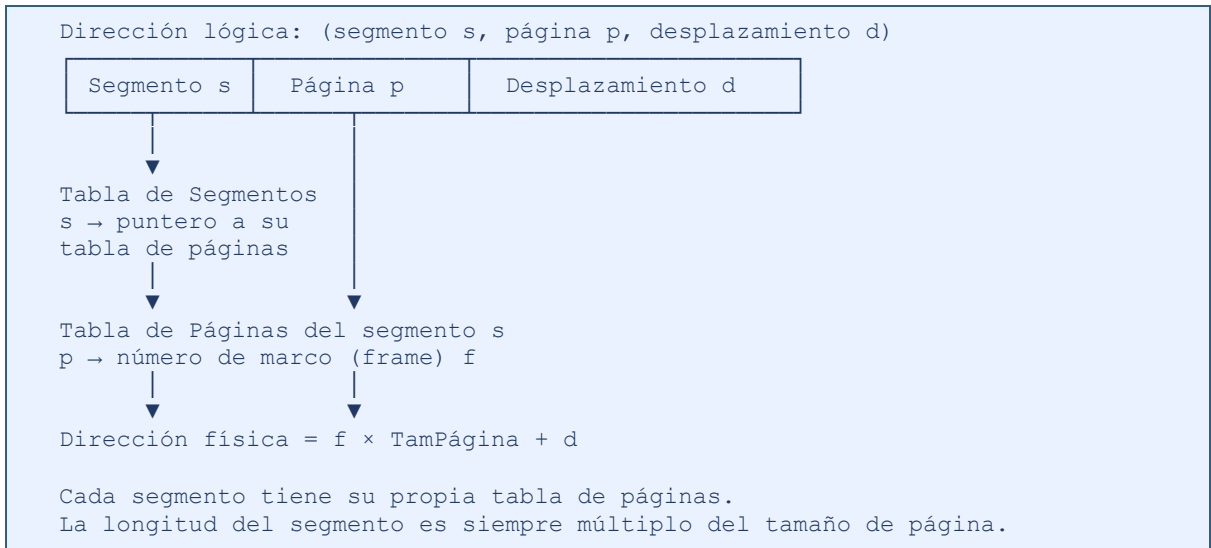
✦ **Nota actualizada:** En la arquitectura x86-64, la segmentación está efectivamente desactivada para el código de usuario (todos los segmentos tienen base 0 y límite máximo). La protección y la separación de memoria se realizan exclusivamente mediante la paginación. La segmentación en x86-64 solo se usa para algunos propósitos específicos del SO (como el acceso a estructuras per-CPU a través del registro FS/GS).

7. Esquemas Combinados: Segmentación Paginada

Los esquemas combinados aprovechan las ventajas de la paginación (sin fragmentación externa) y de la segmentación (organización lógica del espacio de direcciones). El modelo más usado es la segmentación paginada.

7.1 Segmentación Paginada

El espacio lógico está dividido en segmentos (de tamaño variable), y cada segmento internamente se divide en páginas (de tamaño fijo). El resultado es que no hay fragmentación externa (gracias a la paginación) y la organización lógica es clara (gracias a la segmentación).



7.2 Implementación en Arquitecturas Reales

Arquitectura / SO	Esquema de administración de memoria
Intel x86 (32 bits)	Segmentación completa + paginación de 2 niveles. GDT/LDT definen segmentos; CR3 apunta a directorio de páginas.
Intel x86-64 (64 bits)	Segmentación desactivada (flat model). Paginación de 4 niveles (PML4 → PDPT → PD → PT). TLB con soporte PCID.
ARM64 (AArch64)	Solo paginación (3 ó 4 niveles). Sin segmentación. Dos tablas: TTBR0 (usuario), TTBR1 (kernel).

Sistemas Operativos Conferencia 6		Ingeniería Informática · 3er Año
Arquitectura / SO	Esquema de administración de memoria	
Linux (cualquier arq.)	Abstrae el hardware con un modelo unificado de 4/5 niveles de paginación: PGD→P4D→PUD→PMD→PTE.	
Windows (x86-64)	Paginación de 4 niveles. Virtual Address Descriptors (VAD) para gestionar regiones del proceso.	

8. Administración de Memoria en Sistemas Operativos Actuales

8.1 Linux: Gestión de Memoria del Kernel

Componente	Función
Buddy System	Asignación de páginas físicas. Mantiene listas de bloques libres de tamaño 2 ⁿ páginas (n=0..10). /proc/buddyinfo muestra el estado.
SLAB / SLUB Allocator	Asignación eficiente de objetos del kernel de tamaño fijo (inodes, task_struct, PCB...). Elimina la fragmentación interna del Buddy System para objetos pequeños.
vmalloc / mmap	Asignación de regiones virtuales contiguas (aunque físicamente no lo sean). Usado para módulos del kernel y regiones grandes.
Memory-mapped files (mmap)	Mapea ficheros o dispositivos directamente al espacio de direcciones virtual del proceso. Usado para bibliotecas compartidas (.so).
KSM — Kernel Samepage Merging	Fusiona automáticamente páginas físicas idénticas de diferentes procesos (útil en virtualización KVM). Ahorra RAM en sistemas con muchas VMs idénticas.
Huge Pages (THP)	Páginas de 2 MB o 1 GB en lugar de 4 KB. Reduce el tamaño de la tabla de páginas y mejora el TLB hit rate en cargas con grandes datasets.

8.2 Espacio de Direcciones Virtual del Proceso en Linux (x86-64)

Espacio virtual (48 bits usados = 256 TB por mitad):		
Dirección alta (Kernel space)	0xFFFFFFFFFFFFFFFF	Kernel (SO) Código, datos, módulos (zona no mapeada - 128 TB) Stack (crece hacia abajo) Bibliotecas .so (mmap) Heap (crece hacia arriba) Segmento de datos (.data) Segmento de código (.text)
	0xFFFF800000000000	
	[espacio vacío]	
(User space)	0x00007FFFFFFFFFFF	
Dirección baja	0x0000000000000000	

📌 **Nota actualizada:** En Linux, el comando `/proc/<pid>/maps` muestra el mapa completo de memoria virtual de un proceso: regiones de código, datos, heap, stack, bibliotecas mapeadas y su protección (r/w/x). La herramienta `pmap -x <pid>` proporciona un resumen. El comando `free -h` muestra la memoria disponible del sistema; `/proc/meminfo` da el detalle completo incluyendo buffers, caché de páginas y memoria anónima.

9. Estudio Independiente

1. ¿Cuáles son las diferencias entre relocación estática y dinámica? ¿Por qué la relocación dinámica es un prerequisite para el swapping y la multiprogramación con particiones variables?
2. Dada una memoria de 1 MB dividida en unidades de 4 KB: ¿cuántos bits necesita el mapa de bits? ¿Cuántos bytes ocupa? ¿Qué sucede si la unidad asignable se cambia a 8 KB?
3. Compare la paginación y la segmentación en términos de: tipo de fragmentación, visibilidad para el programador, facilidad de compartición y complejidad del hardware. ¿Por qué x86-64 abandonó la segmentación efectiva en favor de la paginación pura?
1. Investigue el concepto de ASLR (Address Space Layout Randomization). ¿Qué vulnerabilidades de seguridad mitiga? ¿Cómo interactúa con la paginación? ¿Está habilitado por defecto en Linux y Windows?
2. ¿Qué es el SLAB allocator de Linux? ¿Por qué se necesita además del Buddy System? ¿Cuál es la diferencia entre SLAB, SLOB y SLUB?

— Fin de la Conferencia 6 —